



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №2
на тему «Діаграма варіантів використання. Сценарії варіантів
використання. Діаграми UML. Діаграми класів. Концептуальна
модель системи.»

Виконала:
Студентка групи ІА-22
Патрик Вікторія

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості.....	3
1. Діаграма варіантів використання (Use Case Diagram)	3
2. Сценарії варіантів використання (Use Case Scenarios).....	3
3. Діаграми UML (Unified Modeling Language)	3
4. Діаграми класів (Class Diagrams).....	4
5. Концептуальна модель системи	4
Тема	5
Схема прецеденту.....	5
Прецеденти та сценарії їх використання.....	6
Прецедент 1: Відкриття файлу	6
Прецедент 2: Збереження файлу	6
Прецедент 3: Використання макросів	6
Діаграма класів	8
Шаблон репозиторію	9
Код на GitHub	10
Структура бази даних	11

Діаграма варіантів використання. Сценарії варіантів використання. Діаграми UML. Діаграми класів.

Концептуальна модель системи.

Мета: проаналізувати тему та намалювати схему прецеденту, діаграму класів, розробити основні класи і структуру системи баз даних.

Хід роботи:

Короткі теоретичні відомості:

1. Діаграма варіантів використання (Use Case Diagram)

Діаграма варіантів використання є графічним представленням функціональності системи з точки зору її користувачів (акторів). Вона відображає:

- **Актори:** Особи або системи, які взаємодіють з вашою системою (наприклад, користувачі, адміністрація).
- **Варіанти використання:** Основні функції або послуги, які система надає акторам (наприклад, реєстрація, авторизація, перегляд даних).
- **Взаємозв'язки:** Лінії, що з'єднують акторів з варіантами використання, показуючи, які функції доступні кожному з акторів.

2. Сценарії варіантів використання (Use Case Scenarios)

Сценарії варіантів використання описують конкретні ситуації, в яких актор взаємодіє з системою для досягнення певної мети. Кожен сценарій включає:

- **Назву:** Коротка та зрозуміла назва сценарію.
- **Актор:** Хто ініціює сценарій.
- **Передумови:** Умови, які повинні бути виконані перед початком сценарію.
- **Основний потік:** Кроки, які описують нормальний сценарій дій.
- **Альтернативні потоки:** Інші можливі шляхи виконання сценарію, які можуть виникнути через помилки або альтернативні дії.

3. Діаграми UML (Unified Modeling Language)

UML — це стандартизована мова моделювання, що використовується для візуалізації, специфікації, конструювання та документування програмних систем. Діаграми UML поділяються на дві основні категорії:

- **Структурні діаграми:** Відображають структуру системи, наприклад, діаграми класів, діаграми компонентів, діаграми об'єктів.

- **Поведенкові діаграми:** Показують динаміку системи, такі як діаграми варіантів використання, послідовності, активності.

4. Діаграми класів (Class Diagrams)

Діаграми класів є основними структурними діаграмами в UML, що показують класи системи, їх атрибути, методи та зв'язки між класами. Основні елементи:

- **Клас:** Показується прямокутником, що містить ім'я класу, атрибути та методи.
- **Атрибути:** Описують дані, які зберігаються в класі (наприклад, ім'я, вік).
- **Методи:** Визначають поведінку класу (наприклад, створити, редагувати).
- **Зв'язки:** Включають асоціації, агрегації, композиції та наслідування, що показують, як класи взаємодіють між собою.

5. Концептуальна модель системи

Концептуальна модель системи — це абстрактне представлення системи, яке підкреслює її основні компоненти і їхні зв'язки, без урахування технічних деталей. Вона зазвичай включає:

- **Сутності:** Основні об'єкти системи (наприклад, користувачі, продукти).
- **Асоціації:** Зв'язки між сутностями, які показують, як вони взаємодіють.
- **Властивості:** Описують характеристики сутностей.

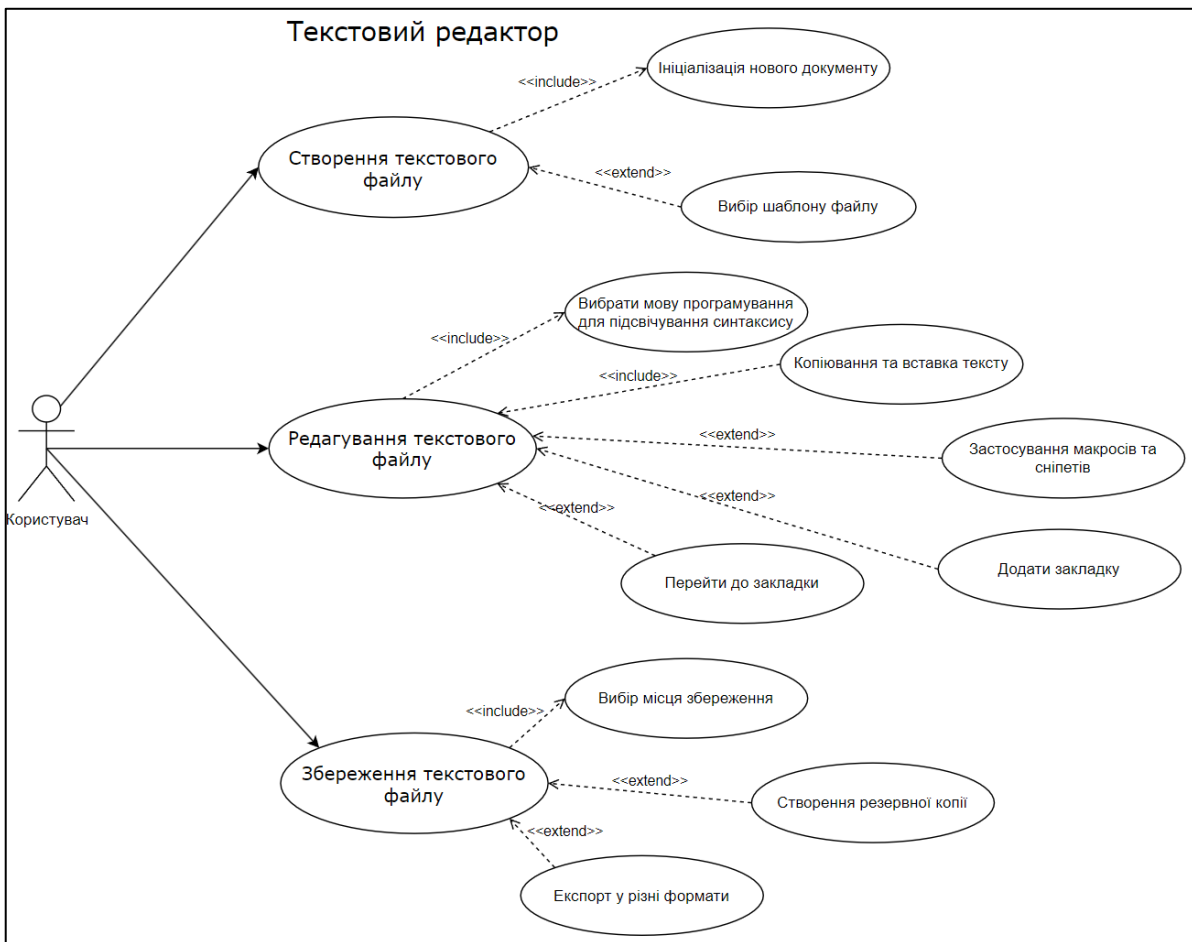
Ця модель допомагає зрозуміти загальну структуру та функціонування системи перед її реалізацією.

Тема:

..3 Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніпшети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Схема прецеденту, що відповідає темі «Текстовий редактор»:



Ця діаграма прецедентів зображує взаємодію користувача з текстовим редактором, відображаючи основні функції, доступні в програмі. Вона демонструє ключові можливості, такі як створення, редагування та збереження текстового файлу. Додаткові прецеденти та розширення включають специфічні дії, що покращують зручність роботи з текстом, наприклад, вибір шаблону, підсвічування синтаксису, використання макросів та сніпсетів, а також створення резервних копій і експорт файлів у різних форматах. Ця діаграма допомагає структурувати функціональні можливості редактора та визначити основні сценарії використання для забезпечення ефективної роботи користувача.

Прецеденти та сценарії їх використання:

Прецедент 1: Відкриття файлу

- **Назва:** Відкриття текстового файлу
- **Передумови:** Користувач запустив текстовий редактор.
- **Післяумови:** Вміст вибраного файлу відкритий у редакторі.
- **Сторони, що взаємодіють:** Користувач, Текстовий редактор.
- **Опис:** Користувач відкриває існуючий текстовий файл для редагування.
- **Основний сценарій:**
 1. Користувач натискає кнопку "Відкрити".
 2. Відкривається діалогове вікно вибору файлу.
 3. Користувач обирає файл і натискає "Відкрити".
 4. Система перевіряє кодування файлу.
 5. Вміст файлу відображається в текстовому редакторі.
- **Виняткові ситуації:**
 - Файл не знайдено: система повідомляє про помилку.
 - Невірне кодування: система запитує підтвердження кодування.
- **Примітки:** Функція відкриття файлу підтримує всі популярні кодування.

Прецедент 2: Збереження файлу

- **Назва:** Збереження текстового файлу
- **Передумови:** Користувач редагує текст у відкритому документі.
- **Післяумови:** Вміст файлу збережений у локальному хранилищі.
- **Сторони, що взаємодіють:** Користувач, Текстовий редактор.
- **Опис:** Користувач зберігає зміни у текстовому файлі.
- **Основний сценарій:**
 1. Користувач натискає кнопку "Зберегти".
 2. Якщо файл уже збережений:
 - Зміни автоматично зберігаються без додаткових дій.
 3. Якщо файл новий (без назви):
 - Відкривається діалогове вікно для введення назви файлу.
 - Користувач вводить ім'я файлу та натискає "Зберегти".
- **Виняткові ситуації:**
 - Відмова від збереження: система запитує підтвердження.
 - Немає прав на запис: система повідомляє про помилку.
- **Примітки:** Можливість збереження у різних форматах (TXT, RTF, інші).

Прецедент 3: Використання макросів

- **Назва:** Використання редагуючого макросу
- **Передумови:** Користувач створив макрос.
- **Післяумови:** Макрос застосовано до тексту в редакторі.
- **Сторони, що взаємодіють:** Користувач, Текстовий редактор.

- **Опис:** Користувач виконує раніше створений макрос для автоматизації задач.
- **Основний сценарій:**
 1. Користувач натискає кнопку "Макроси".
 2. Відображається список доступних макросів.
 3. Користувач обирає потрібний макрос.
 4. Система виконує макрос і відображає результат.
- **Виняткові ситуації:**
 - Макрос не знайдено: система повідомляє про помилку.
 - Виникає помилка під час виконання: система виводить повідомлення про помилку.
- **Примітки:** Користувач може редагувати макроси після їх створення.

Діаграма класів:



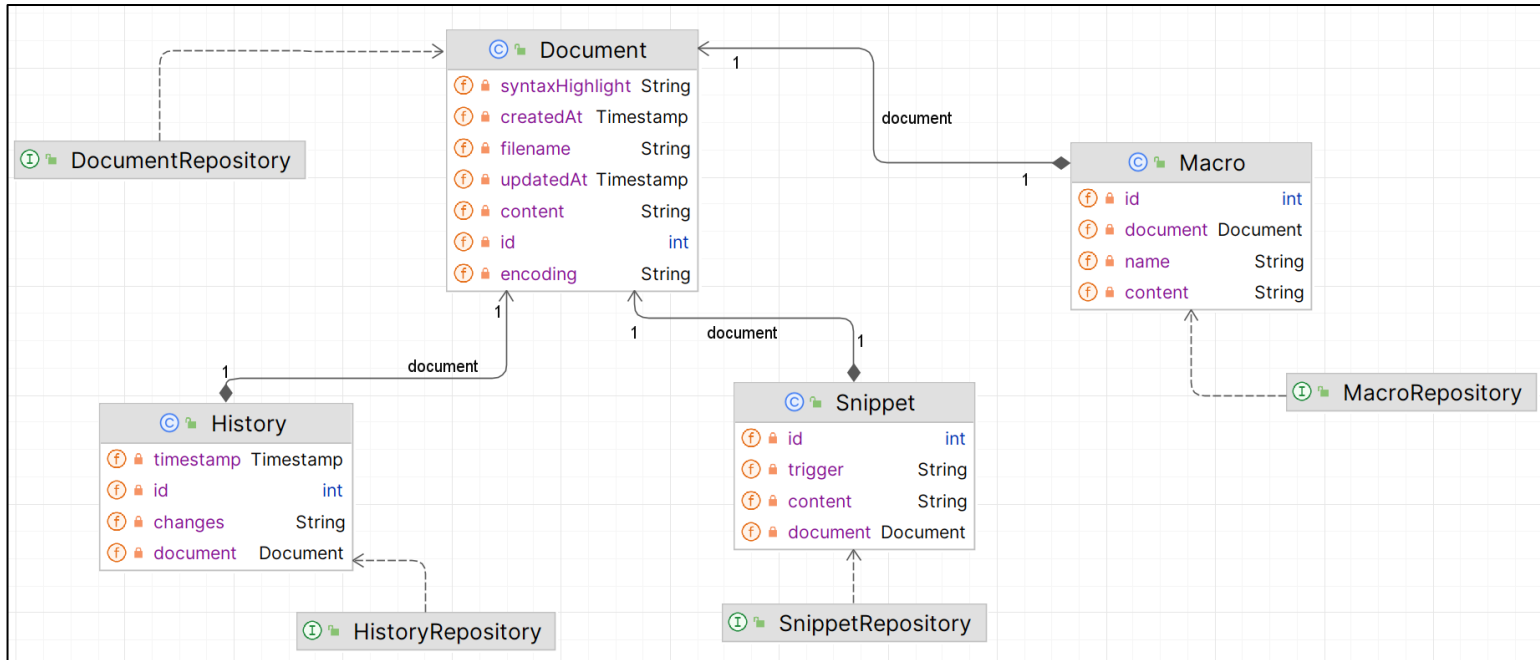
Ця діаграма класів відображає архітектуру текстового редактора з використанням кількох компонентів для управління документами, історією, макросами та фрагментами. Діаграма містить такі основні елементи:

1. **DocumentController, MacroController, SnippetController, HistoryController** – це контролери, які керують відповідними функціональними частинами текстового редактора, такими як документи, макроси, фрагменти та історія. Вони взаємодіють із сервісами для виконання CRUD операцій (створення, читання, оновлення та видалення).
2. **DocumentServiceImpl, MacroServiceImpl, SnippetServiceImpl, HistoryServiceImpl** – це реалізації сервісних класів, які виконують основну бізнес-логіку для кожної з відповідних сутностей. Вони використовують репозиторії для зберігання та отримання даних.
3. **DocumentService, MacroService, SnippetService, HistoryService** – це сервіси, що визначають основні методи для маніпулювання даними (документи, макроси, фрагменти та історія), такі як створення, оновлення, видалення, отримання за ідентифікатором та отримання всіх записів.
4. **DocumentRepository, MacroRepository, SnippetRepository, HistoryRepository** – це репозиторії, які забезпечують доступ до бази даних для кожної з сутностей.

5. **TextEditorApplication** – головний клас програми з методом main, що ініціалізує текстовий редактор і всі його компоненти.

Взаємозв'язки між контролерами, сервісами та репозиторіями показують потік даних та залежності між компонентами програми.

Шаблон репозиторію:



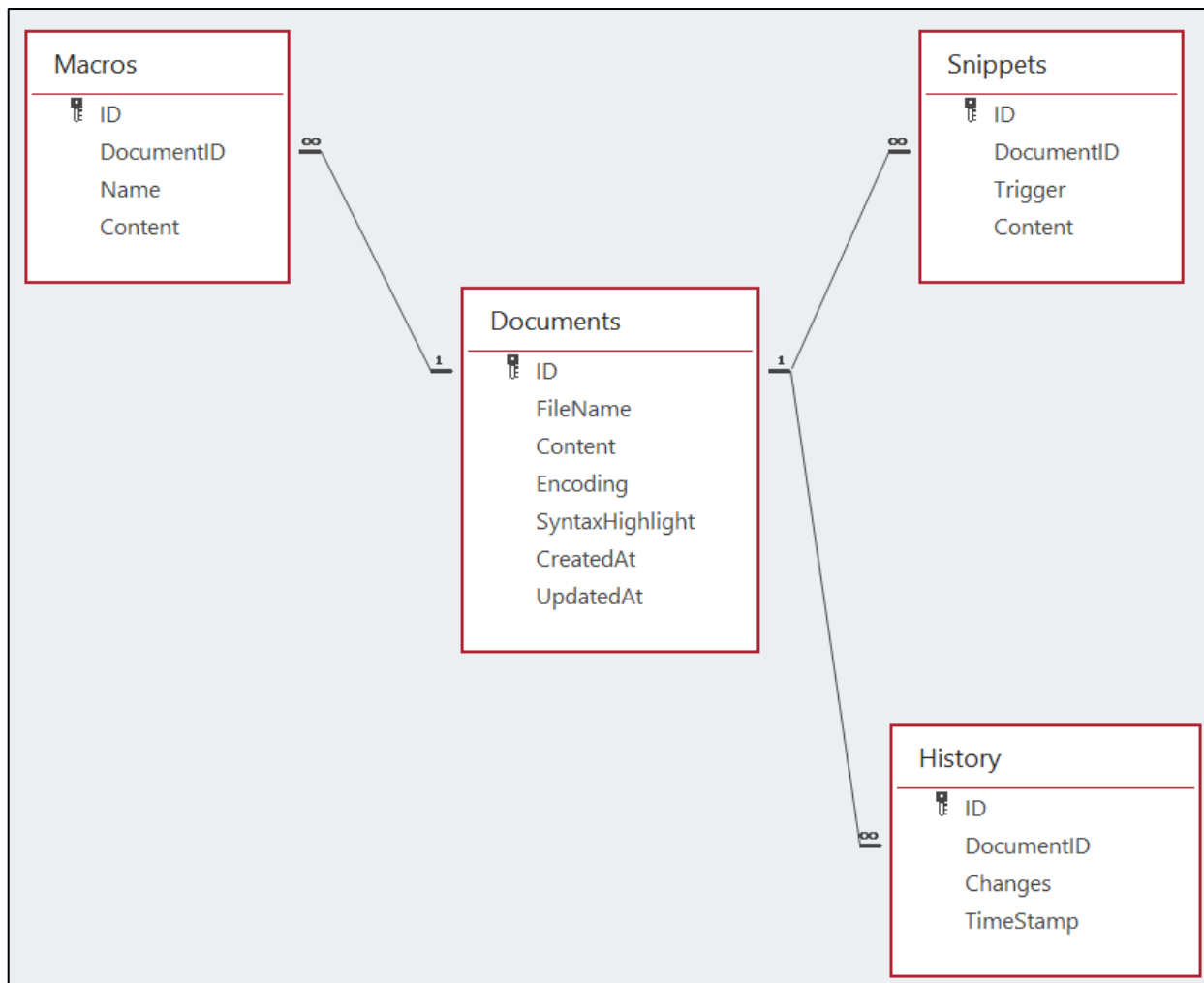
Цей шаблон репозиторію представляє модель текстового редактора, що включає кілька основних сутностей та їх взаємозв'язки:

1. **Document** – центральна сутність, яка представляє документи, що редагуються в текстовому редакторі. Всі інші сутності мають зв'язок із документом.
2. **History** – зберігає інформацію про зміни, зроблені в документі. Кожна історія змін пов'язана з одним конкретним документом, показуючи зв'язок "один до одного".
3. **Macro** – макроси, які можуть бути застосовані до документа. Кожен макрос пов'язаний з одним документом, дозволяючи автоматизувати повторювані дії.
4. **Snippet** – фрагменти тексту або коду, що також пов'язані з документом. Ці фрагменти можуть використовуватися для швидкого вставлення частин тексту в документ.
5. **Репозиторії** (**DocumentRepository**, **HistoryRepository**, **MacroRepository**, **SnippetRepository**) – відповідають за збереження та управління даними для відповідних сутностей.

Діаграма ілюструє зв'язки між документами та такими елементами, як історія змін, макроси та фрагменти, підкреслюючи їхню взаємодію в межах одного документа.

Код на GitHub: <https://github.com/vvvvvvvvvviktoriia/TextEditor>

Структура бази даних:



База даних складається з чотирьох основних таблиць, які мають тісні зв'язки між собою, що забезпечує ефективне управління документами та їх редагуванням.

1. **documents:** Ця таблиця є основною і містить інформацію про текстові файли, такі як ім'я файлу, вміст, кодування та тип підсвічування синтаксису. Вона також має поля для відстеження дати створення та останнього оновлення документа. Всі інші таблиці взаємодіють з нею через зовнішній ключ `document_id`, що робить її центром даних.
2. **macros:** Таблиця макросів дозволяє зберігати автоматизовані дії для редагування документів. Кожен макрос має унікальний ідентифікатор і пов'язаний з конкретним документом за допомогою зовнішнього ключа `document_id`. Це дозволяє швидко застосовувати визначені макроси до відповідних документів.
3. **snippets:** У цій таблиці зберігаються шаблони тексту, які можна швидко вставити у документи. Кожен сніппет має тригер для вставлення, а також пов'язується з конкретним документом через `document_id`. Це

забезпечує швидкий доступ до часто використовуваних фрагментів коду чи тексту.

4. **history:** Таблиця історії редагування зберігає записи про всі зміни, внесені до документів. Це дозволяє відстежувати кожну правку разом із її часом і описом змін, що може бути корисним для відновлення попередніх версій або аналізу історії редагування.

Загалом, така структура бази даних дозволяє ефективно організувати та управляти документами, макросами, сніпетами та історією редагування, забезпечуючи легкий доступ та зручність використання. Зв'язки між таблицями дозволяють підтримувати цілісність даних і забезпечують можливість їх швидкої обробки.

Висновок: завдяки цій лабораторній роботі, я проаналізувала тему своєї роботи та намалювала схему прецеденту, діаграму класів, розробила основні класи і структуру системи баз даних.